```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
```

```
np.random.seed(42)
n = 200
TV = np.random.uniform(0, 300, n)
Radio = np.random.uniform(0, 50, n)
Newspaper = np.random.uniform(0, 100, n)
noise = np.random.normal(0, 3, n)
Sales = 4 + 0.045*TV + 0.19*Radio + 0.01*Newspaper + noise

df = pd.DataFrame({'TV': TV, 'Radio': Radio, 'Newspaper': Newspaper, 'Sales': Sales})
df.head()
```

	TV	Radio	Newspaper	Sales
0	112.362036	32.101582	10.312387	13.667943
1	285.214292	4.206998	90.255291	16.157907
2	219.598183	8.081436	50.525237	15.601552
3	179.597545	44.927709	82.645747	18.338885
4	46.805592	30.321453	32.004960	10.526429

Next steps: Generate code with df | New interactive sheet

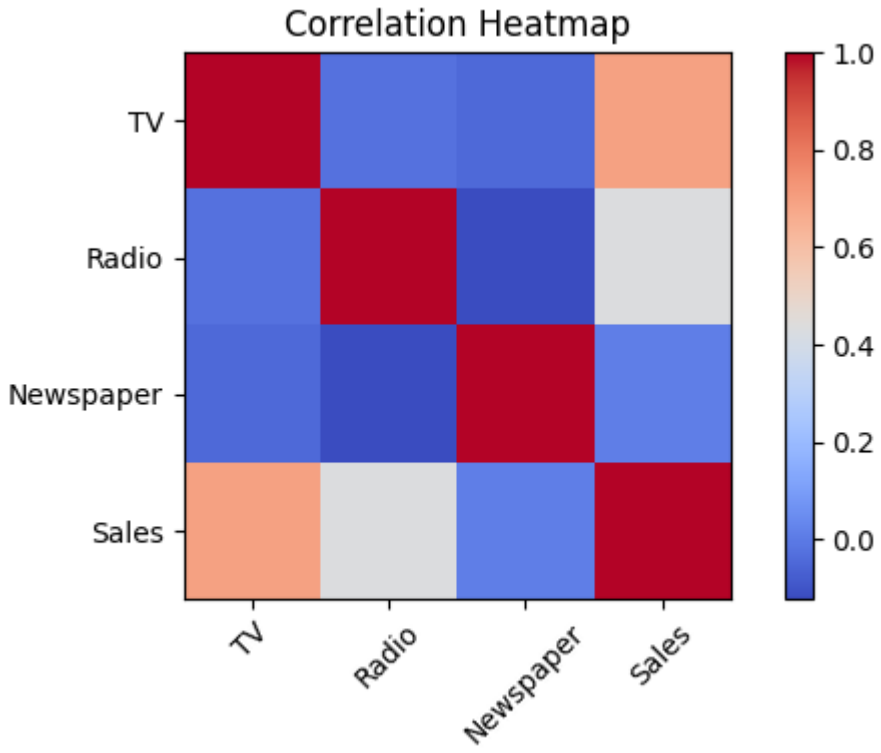
```
print(df.info())
print(df.describe())
print("Missing values:\n", df.isnull().sum())

print("\nCorrelation with Sales:\n", df.corr()['Sales'])

plt.figure(figsize=(6, 4))
plt.imshow(df.corr(), cmap='coolwarm', interpolation='none')
plt.colorbar()
plt.xticks(range(len(df.columns)), df.columns, rotation=45)
plt.yticks(range(len(df.columns)), df.columns)
plt.title('Correlation Heatmap')
plt.tight_layout()
plt.show()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 4 columns):
 #   Column     Non-Null Count  Dtype  
---  ------     --------------  -----  
 0   TV         200 non-null    float64
 1   Radio      200 non-null    float64
 2   Newspaper  200 non-null    float64
 3   Sales      200 non-null    float64
dtypes: float64(4)
memory usage: 6.4 KB
None
               TV       Radio   Newspaper       Sales
count  200.000000  200.000000  200.000000  200.000000
mean   145.201870   25.218757   52.067278   15.667832
std     88.467429   14.650060   30.730879    5.523723
min      1.656635    0.253079    1.083765   -1.055002
25%     68.574724   13.073125   25.552032   11.374770
50%    148.345876   27.081994   52.539948   16.191591
75%    227.057885   37.109966   81.111489   19.841205
max    296.066081   49.525257   99.971767   28.169749
Missing values:
 TV           0
Radio        0
Newspaper    0
Sales        0
dtype: int64

Correlation with Sales:
 TV           0.696337
Radio        0.431924
Newspaper    0.011870
Sales        1.000000
Name: Sales, dtype: float64
```




```
X = df[['TV', 'Radio', 'Newspaper']]
y = df['Sales']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=
print("Train size:", X_train.shape, "Test size:", X_test.shape)
```

```
Train size: (160, 3) Test size: (40, 3)
```

```
model = LinearRegression()
model.fit(X_train, y_train)
```

```
▾ LinearRegression
LinearRegression()
```

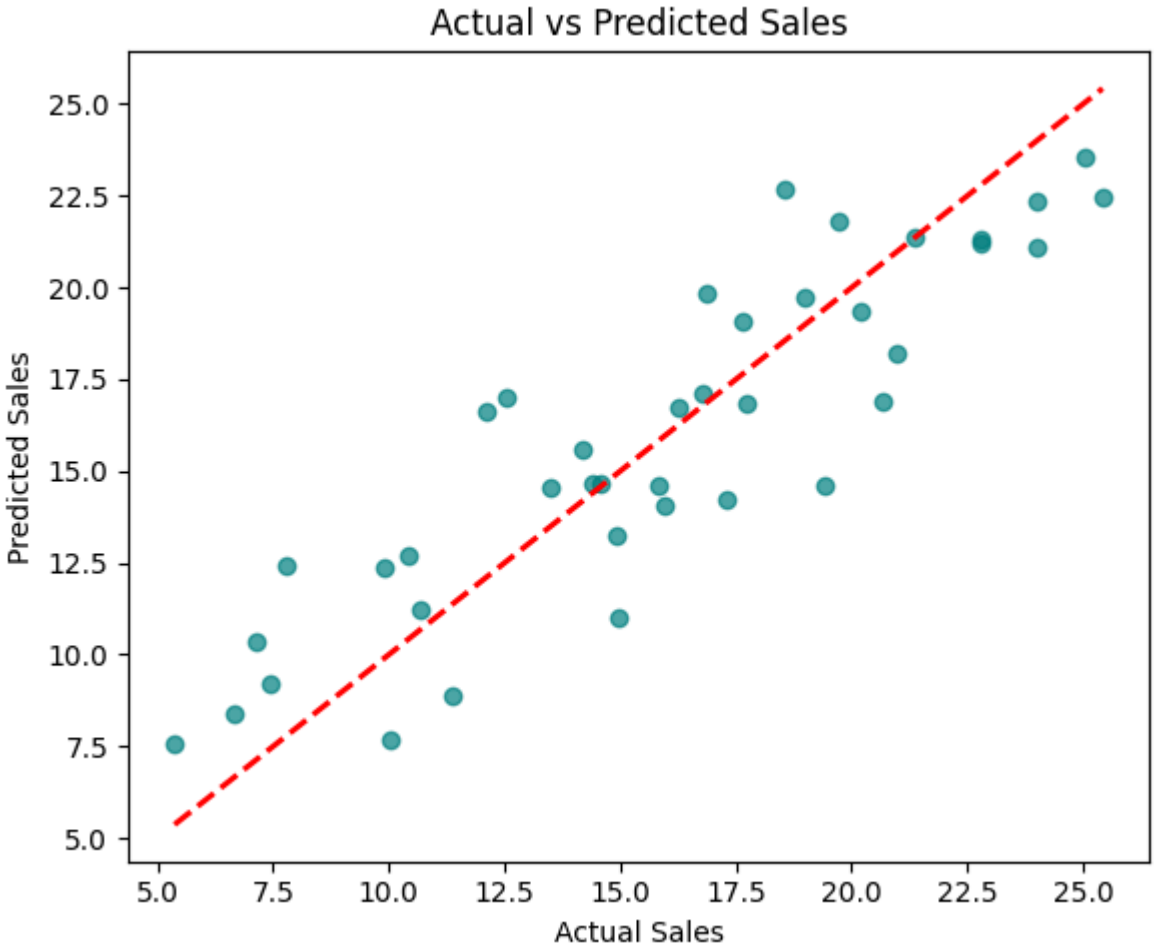
```
y_pred = model.predict(X_test)

results_df = pd.DataFrame({'Actual': y_test.values, 'Predicted': y_pred})
results_df.head(10)
```

	Actual	Predicted
0	14.168948	15.601237
1	7.450818	9.208666
2	17.626502	19.065608
3	17.735337	16.837319
4	10.034775	7.686447
5	21.348299	21.385130
6	22.787115	21.316796
7	19.398059	14.593027
8	14.415663	14.652487
9	18.561927	22.696961

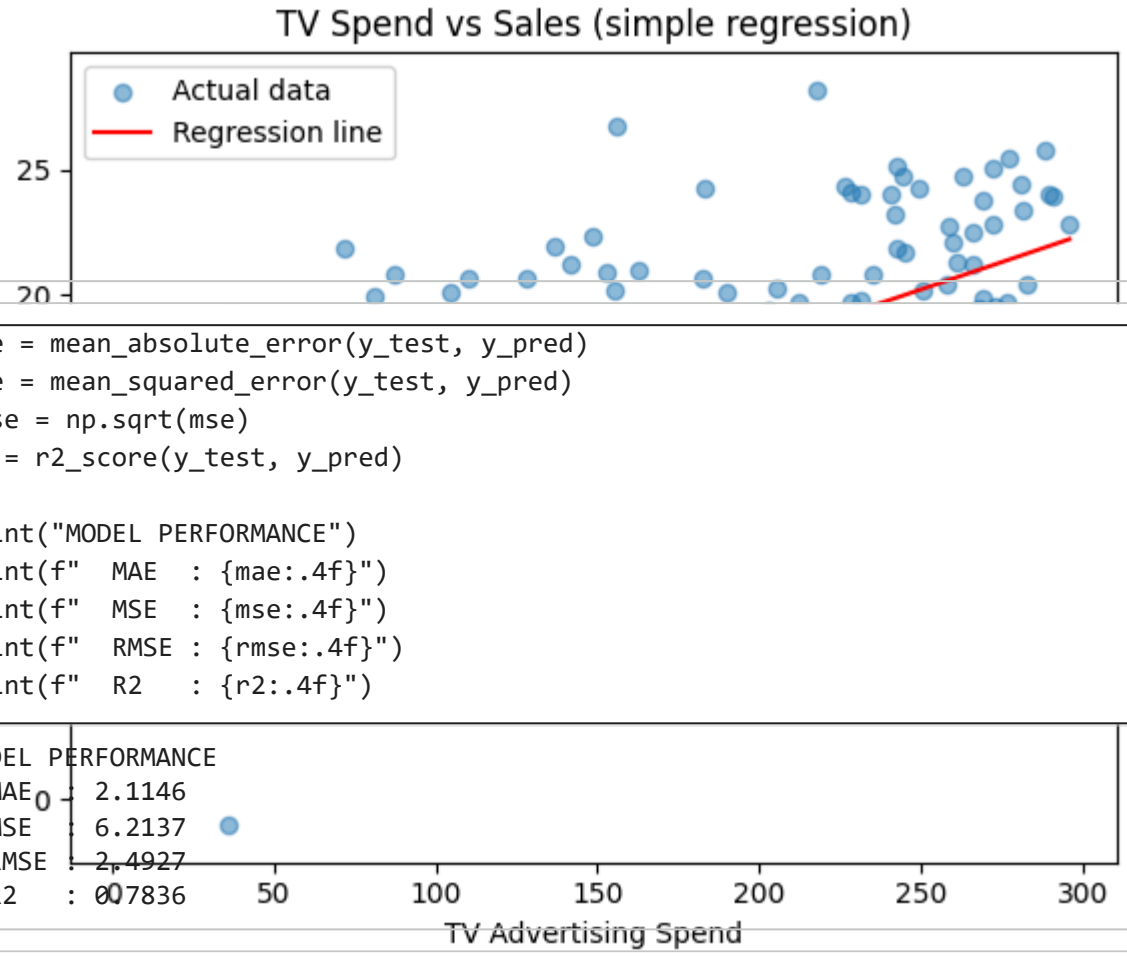
Next steps: Generate code with `results_df` New interactive sheet

```
plt.figure(figsize=(6, 5))
plt.scatter(y_test, y_pred, alpha=0.7, color='teal')
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw=2)
plt.xlabel('Actual Sales')
plt.ylabel('Predicted Sales')
plt.title('Actual vs Predicted Sales')
plt.tight_layout()
plt.show()
```




```
plt.figure(figsize=(6, 5))
plt.scatter(df['TV'], df['Sales'], alpha=0.5, label='Actual data')
single_model = LinearRegression().fit(df[['TV']], df['Sales'])
x_line = np.linspace(df['TV'].min(), df['TV'].max(), 100).reshape(-1, 1)
plt.plot(x_line, single_model.predict(x_line), color='red', label='Regression line')
plt.xlabel('TV Advertising Spend')
plt.ylabel('Sales')
plt.title('TV Spend vs Sales (simple regression)')
plt.legend()
plt.tight_layout()
plt.show()
```

```
/usr/local/lib/python3.13/dist-packages/sklearn/utils/validation.py:2739: UserWarning:
  warnings.warn(
```




```
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("MODEL PERFORMANCE")
print(f"  MAE  : {mae:.4f}")
print(f"  MSE  : {mse:.4f}")
print(f"  RMSE : {rmse:.4f}")
print(f"  R2   : {r2:.4f}")
```

```
MODEL PERFORMANCE
  MAE  : 2.1146
  MSE  : 6.2137
  RMSE : 2.4927
  R2   : 0.7836
```

```
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("MODEL PERFORMANCE")
print(f"  MAE  : {mae:.4f}")
print(f"  MSE  : {mse:.4f}")
print(f"  RMSE : {rmse:.4f}")
print(f"  R2   : {r2:.4f}")
```

```
MODEL PERFORMANCE
  MAE  : 2.1146
  MSE  : 6.2137
  RMSE : 2.4927
  R2   : 0.7836
```